

Major Frontend Migration for GRNsight, a Web Application for
Visualizing Gene Regulatory and Protein-Protein Interaction Network Models

Cecilia J. Zaragoza

Department of Computer Science, Loyola Marymount University

CMSI 4081/4082: Senior Thesis

Dr. John David N. Dionisio

Dr. Kam D. Dahlquist

May 13, 2026

Abstract

A gene regulatory network (GRN) consists of genes, transcription factors, and the regulatory connections between them which govern the level of expression of mRNA and protein from genes. A protein-protein physical interaction network (PPI) represents a set of proteins that bind to each other. A specialized software tool would help biologists better view and analyze such networks, so we created GRNsight, an open-source web application for visualizing models of GRNs and PPIs, either uploaded by a user or obtained from our backend budding yeast database. Graph models display rectangular nodes, representing genes/proteins, with edges connecting them. GRNs have directed red edges with pointed end markers or blue edges with blunt end markers to indicate regulatory relationships (activation or repression, respectively), with edge thickness indicating regulation strength. PPIs have undirected black edges to indicate physical binding. Time-course expression data can be displayed on GRN nodes to facilitate understanding of the regulatory relationships that generated that pattern of expression, creating new insights. GRNsight has been in development for over a decade, during which time both biological datasets and web technologies have advanced significantly. While the original implementation successfully supports the core functionality, it relies on older tools that make long-term maintenance and the addition of new features increasingly difficult. To address this challenge, GRNsight's JavaScript codebase has been migrated from a jQuery/Bootstrap/Pug framework to a modern React-based frontend which improves code readability, state management, and allows for more advanced interactive features. By modernizing the underlying framework, GRNsight is now positioned to continue expanding its functionality as new visualization needs arise for researchers to gain deeper insights into complex molecular systems. GRNsight React is available at <https://dondi.github.io/GRNsight/react-thesis-4081>.

Introduction

Biological Networks

Cells contain thousands of genes that encode all of the information needed for the cell to survive, divide, differentiate, and respond to environmental cues. Gene expression is defined as the flow of information in a cell from DNA to RNA to protein to convert the information encoded in a gene (DNA) into a cell function (protein) (Brody, n.d.). Gene expression requires an “on/off” switch to control when and where RNA and proteins are made and a “volume control” to determine how much of those products are made (Brody, n.d.). The gene expression process is carefully regulated and changes greatly under different conditions and cell types to determine what functions a cell can perform at a given time (Brody, n.d.).

Figure 1A: Transcription factor binding to DNA in a gene regulatory network.

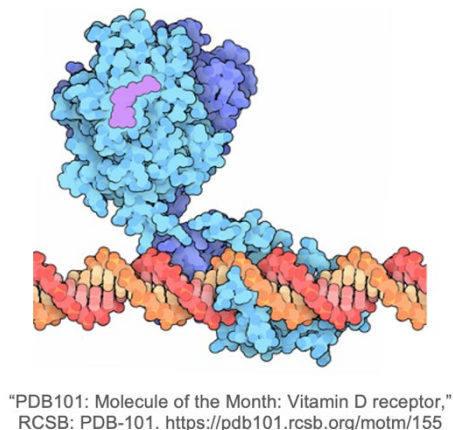
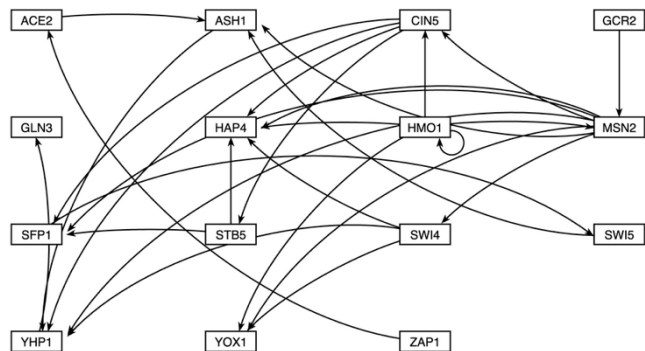


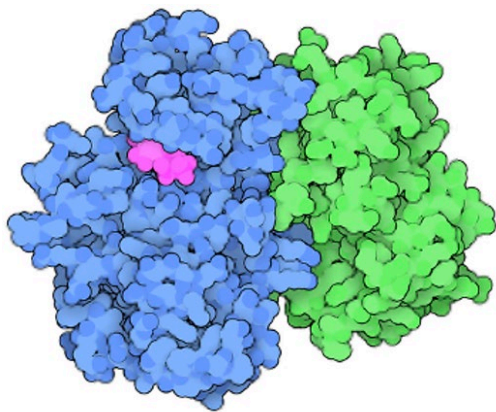
Figure 1B: Grid layout of a gene regulatory network in GRNsight.



Proteins called regulatory transcription factors bind to DNA to regulate the expression of particular genes (Figure 1A). For example, in budding yeast, *Saccharomyces cerevisiae*, approximately 250 regulatory transcription factors control the expression of the approximately 6000 genes in the yeast genome. A single transcription factor can regulate tens to thousands of genes; a single gene can be regulated by one or up to a dozen regulatory transcription factors. A

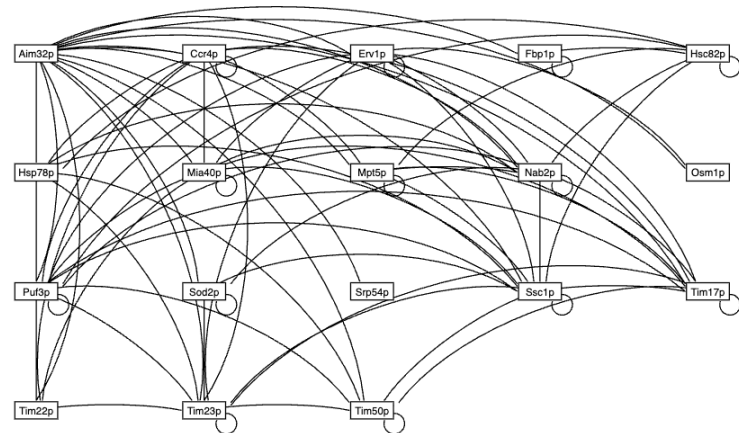
gene regulatory network (GRN) is thus defined as the genes, transcription factors, and the regulatory connections between them (Figure 1B). Transcription factors are encoded by genes that are regulated by other transcription factors, and some genes are autoregulatory where the transcription factor it encodes regulates its own gene.

Figure 2A: Two proteins binding to each other in a protein-protein interaction.



"PDB101: Molecule of the Month: Cyclin and cyclin-dependent kinase,"
RCSB: PDB-101. <https://pdb101.rcsb.org/motm/236>

Figure 2B: Figure 1B: Grid layout of a protein-protein interaction in GRNsight.



A protein-protein physical interaction (PPI) is defined as the physical contact (binding) between two or more proteins (Figure 2A). Protein binding is the basis for nearly all cell functions. A PPI network represents multiple binding interactions required to carry out complex biological functions (Figure 2B) (Farooq, n.d.). GRNs can be mathematically modeled by applications such as our sister project, GRNmap, a MATLAB program that given input degradation rates and expression data, estimates production rate, expression threshold, and regulatory weight parameters and performs forward simulations of an ordinary differential equations model of a small GRN (Dahlquist et. al, 2015; Dahlquist et. al, 2016). Computer representations of GRNs, such as the models output by GRNmap, are in the form of a tabular spreadsheet (adjacency matrix) that is not easily interpretable by biologists (Dahlquist et. al, 2016). Ideally, GRNs should be displayed as diagrams (graphs) detailing the regulatory

relationships (directed edges) between each gene (node) in the network (Dahlquist et. al, 2016).

To address this need, we developed GRNsight, a web application for viewing small-scale models of GRNs and PPIs, optimized for the display of fewer than 35 nodes. GRNsight development began in the Spring of 2014 and has undergone continuous development since that time conducted by successive generations of students (Dahlquist et. al, 2016).

GRNsight as a Visualization Tool

Figure 3: GRNsight graphical user interface and menus.

The screenshot displays the GRNsight web application interface. At the top, a menu bar includes Network, Layout, Node, Edge, View, Export, Help, and Demo. The main window shows a network graph with nodes (genes) and directed edges (regulatory interactions). The nodes are labeled with gene names like ACE2, ASH1, CIN5, GCR2, GND3, HAP4, HMO1, MSA2, SFP1, STB5, SWH1, SWI5, YOX1, and ZAP1. Edges are color-coded and labeled with numerical values representing interaction strengths. A key in the bottom right corner identifies the symbols: a blue box for 'gene', a red arrow for 'activation', a blue arrow for 'repression', and a dashed line for 'weak influence'.

On the left side, there are six numbered panels for user controls:

- 1. Network**
 - Demo files are provided.
 - Can import and export Excel, SIF, or GraphML files.
 - Can construct a network from SGD regulation or PPI data stored in a backend database.
- 2. Layout**
 - Users can select a force graph layout or grid layout.
 - Force graph parameter sliders:
 - Link distance determines the minimum distance between nodes.
 - Nodes have a charge which repels or attracts other nodes.
- 3. Node**
 - Coloring can be based on user-uploaded expression data or from published expression datasets stored in a backend database.
 - Datasets available:
 - Apweiler et al. 2012, GSE33098
 - Barreto et al. 2012, GSE24712
 - Dahlquist et al. 2018, GSE83656
 - Kitagawa et al. 2002, GSE9336
 - Thorsen et al. 2007, GSE6068
- 4. Edge**
 - Edge coloring can be toggled on and off.
 - Buttons enable the user to always/never see edge weights or see them upon mouseover.
 - Allows user to set normalization factor so that edge thicknesses for different graphs can be rendered on the same scale.
 - Changing the gray threshold deemphasizes weaker edges in the visualization.
 - Presenting gray edges as dashed lines further deemphasizes weak edges.
- 5. View**
 - Multiple viewport sizes available.
 - Graph bounding box can be separated from viewport.
 - Zooming and scrolling available.
- 6. Export**
 - Basic data export to SIF or GraphML or full GRNmap-compatible Excel workbook, with yeast mRNA degradation rates from Neymotin et al. (2014).
 - Image export to PNG, SVG, or PDF.

The right side of the interface contains a large network visualization area with a toolbar for zooming and panning. Below the network, there are additional controls for 'Node' (Enable Node Coloring, Select from user-uploaded expression data, Top Dataset: Dahlquist_2018_wt, Bottom Dataset: Same as Top Dataset, Average Replicate Values, Log Fold Change Max Value: 3) and 'Edge' (Enable Edge Coloring, Hide/Show Edge Weights, Always Show Edge Weights, Edge Weight Normalization Factor: 5.212, Gray Threshold: 5%, Show Gray Edges as Dashed).

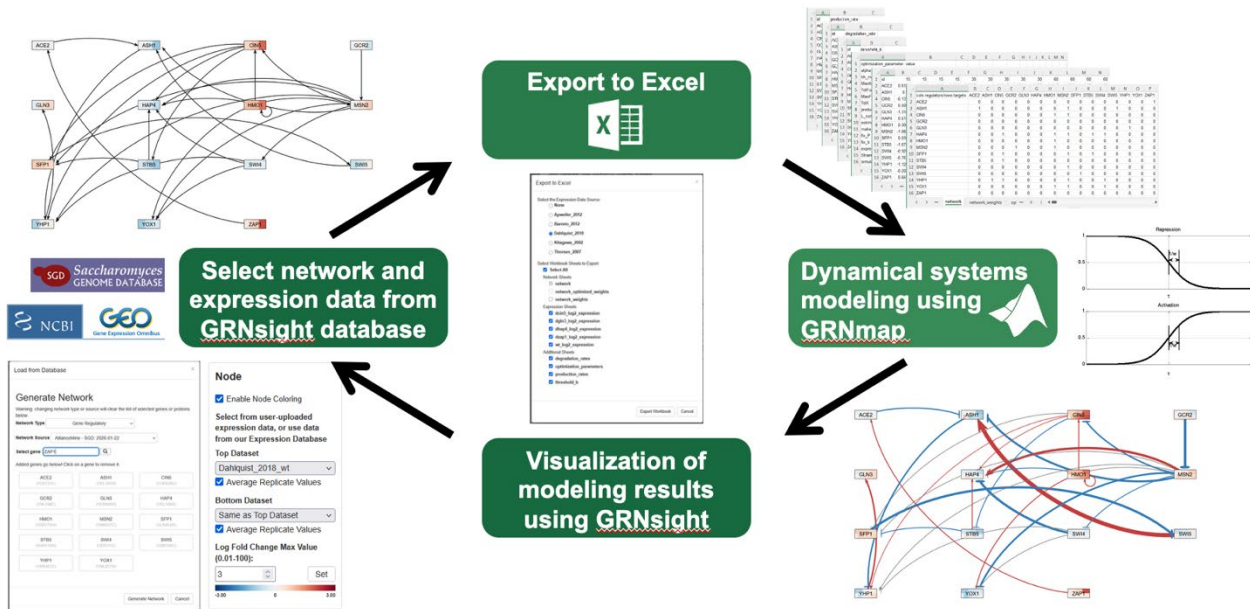
GRNsight, pictured in Figure 3, is an integral part of a data analysis workflow for biologists who model GRNs (Figure 4). GRNsight allows users to upload Excel workbooks generated by GRNmap or Excel, SIF, or GraphML files created by the user according to our Documentation (Dahlquist et. al, 2016). Users without their own network data can load a GRN

by selecting genes from the backend PostgreSQL network database that contains regulation data from the *Saccharomyces* Genome Database (SGD), and demo files are also available (Dahlquist et. al, 2016). GRNsight automatically creates and displays a graph of the GRN model through a physics-based force simulation based on our customizations to the D3.js library or in a grid layout (Dahlquist et. al, 2016; *What Is D3?*, 2024). Users can change the force graph parameters with sliders for link distance, which determines the minimum distance between nodes, and charge, which attracts or repels other nodes. The application colors the edges, adjusts their thickness, and applies arrowhead or blunt-end markers based on the sign (activation or repression) and the strength (magnitude) of the regulatory relationship (weight parameter), respectively. Edge coloring can be toggled on or off, and users can set an edge weight normalization factor to control the thickness of the lines. Edges that have a weight value within 5% of 0 are shown as gray; users can control the gray threshold with a slider or choose to show gray edges as dashed lines. Nodes can be colored based on time course gene expression data supplied in the input workbook or from select public yeast datasets stored in the backend PostgreSQL expression database (Dahlquist et. al, 2016). Node coloring can be toggled on and off, and the user can set their preferred log fold change max value.

GRNsight allows the user to modify the graph display by panning, zooming, and moving nodes to define the best visual layout for the network (Dahlquist et. al, 2016). The viewport size can be changed (small, medium, large, or fit to window); a “restrict graph to viewport” checkbox prevents nodes from drifting beyond the edges of the viewport. Graph images can be printed or exported to .png, .jpg, and .svg formats. Graph data can be exported to SIF, GraphML, or Excel formats. GRNsight can also display PPI networks with undirected edges, either uploaded by the user or loaded from our backend PostgreSQL network database which contains protein-protein

physical interaction data from SGD (Dahlquist et. al, 2016). PPIs have more limited features than GRNs. All user controls for GRNsight can be accessed via side panels or the navigation bar, except for the Export and Help functions which are only found in the navigation bar. The Help menu points users to our Documentation webpage and developer wiki.

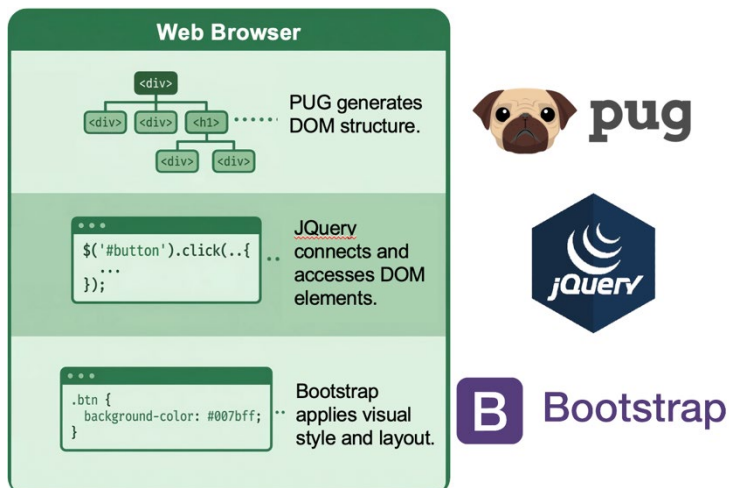
Figure 4: There are four steps in the main user interaction flow in GRNsight.



Materials and Methods

Inspiration for Migration

Figure 5: Pug, jQuery, and Bootstrap were the frontend frameworks used in Classic.



Development on GRNsight began in Spring 2014, so it was built with the tools that were standard for web development at the time: Pug, jQuery, and Bootstrap (Figure 5; Methvin, 2014; Bhattacharyea, 2025). Pug files are templates that generate the DOM structure for the web application. Once Pug files are loaded into a web browser, the template files are compiled into static HTML files. jQuery, a JavaScript library, must be used to connect and access the DOM elements, which includes reading any user interaction with different elements, sharing data values across the app, and making any user interface updates to the web application. jQuery can access any value across the app by targeting elements by ID, class, or tag name. Bootstrap is a free, open-source frontend framework that contains pre-written CSS, HTML, and JavaScript components. To override styles, GRNsight developers can write their own custom CSS classes.

The Pug/jQuery/Bootstrap framework was industry-standard when GRNsight began development in 2014 when it was used by over 61 percent of the top 100,000 web applications (Methvin, 2014). However, continued use of the framework has resulted in code redundancy and complex UI updates today, especially since GRNsight's feature list has grown over time. The use of Pug and jQuery divides the logic between the structure of the pages and updates to the pages, which means the developer has to track changes across multiple layers of the codebase, making it harder to track down bugs as well. jQuery was initially developed and gained popularity because it allowed cross-browser compatibility for event listeners, event handling, easier CSS manipulation, and AJAX for asynchronous features (Bhattacharyea, 2025). However, jQuery is an imperative framework, meaning it requires a lot of manual work by the developer to track when elements are interacted with, how data is shared across the app, and deciding when the app should be re-rendered. The developer must manually code listeners to see when the user enters values in text input fields, moves sliders, or clicks on buttons. There is also no built-in,

centralized way to store data in the web application using jQuery, so the developer must decide where data is shared throughout the app through data attributes on individual elements. These characteristics of Pug and jQuery result in manual updates by the developer which could be error-prone and lead to an increased amount of redundant code. To prevent these issues in the Pug/jQuery/Bootstrap framework, there was a need to migrate GRNsight to a new frontend framework. The version of GRNsight with the Pug/jQuery/Bootstrap framework is now referred to as GRNsight Classic.

GRNsight Architecture

Figure 6: In the GRNsight architecture, there is a separation of concerns between the frontend and the backend, which is denoted with the green dashed line. The backend includes the web API server, the web application server, and the PostgreSQL database. The web API that accepts files in Microsoft Excel workbook (.xlsx), SIF (.sif), and GraphML formats and converts them into a unified JSON representation. The web application server provides code and resources for the graphical user interface. The frontend includes the graphical user interface that displays the JSON representation of the graph.

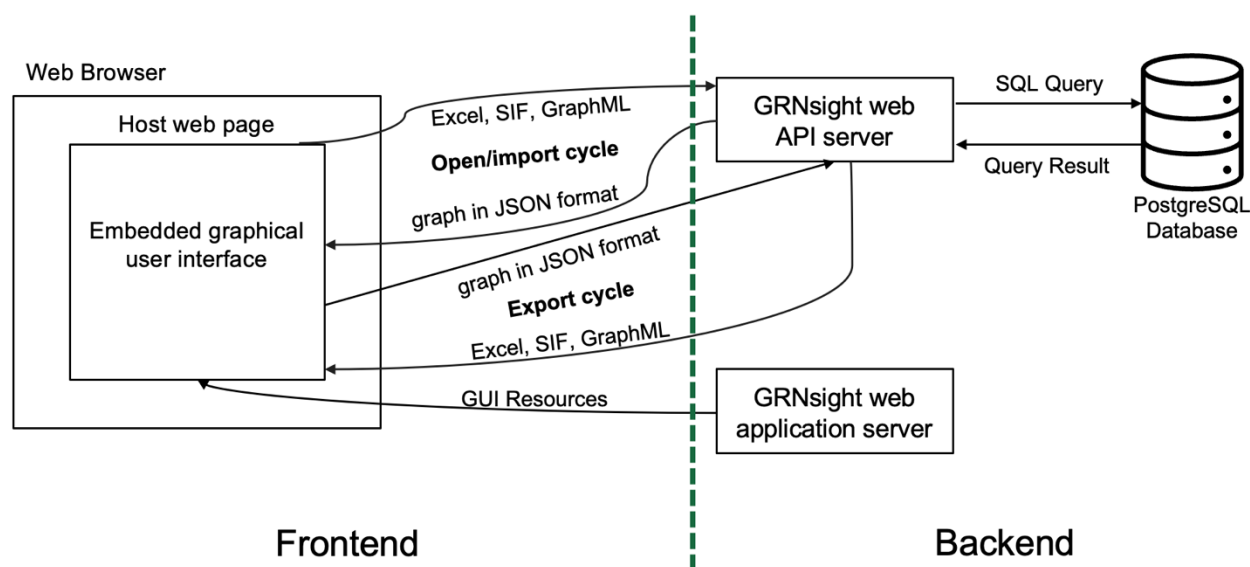


Figure 6 shows the current GRNsight architecture. In a web application, the frontend is everything the user sees and can interact with. GRNsight's frontend includes the web browser,

the host webpage, and the embedded graphical user interface. The host web page is a page served by GitHub Pages (gh-pages) that contains the GRNsight header graphic and navigation (side and bottom links) between different pages of the website (Home, About, Documentation, People, Links, Beta). The embedded graphical user interface displays the navigation bar, sidebar, graph, and graph controls. Users can view two versions of GRNsight: Home (production: main branch), which is the most stable version of GRNsight, typically updated 1-2 times per year, or Beta (beta branch), which contains the most recent and experimental features (typically updated weekly).

The backend includes all the code and services that may not necessarily be visible to the user but that make the frontend functional (Figure 6). There are two main user flows in GRNsight: the open/import cycle and export cycle (Figure 6). In the open/import cycle, users can upload an Excel, SIF, or GraphML file to the GRNsight web API server which converts the file to a JSON representation so that it can be visualized. If the user would like to load a network for *Saccharomyces cerevisiae* from genes/proteins stored in our Network Database, then the web API will make an SQL Query request to GRNsight's PostgreSQL database for the genes/proteins input by the user in the Load from Database modal. If the user would also like to display node coloring with gene expression data from our Expression Database, then the web API will make an SQL Query request to GRNsight's PostgreSQL database. There are four main tables in GRNsight's database: grnsettings, gene_expression, gene_regulatory_network_with_timestamp, and protein-protein_interactions_with_timestamp.

In the export cycle, the user can export a graph that they are currently viewing. The frontend sends the graph in a JSON format to the web API server which converts that to the requested file format of SIF, GraphML, or Excel. If the user is exporting a graph with data from the Network or Expression databases, then the web API makes an SQL query to the database for

those genes/proteins. The graphical user interface is served up by the GRNsight web application server in the backend, which contains all its related code and resources. Because of the separation of concerns between the frontend and the backend, we can migrate the frontend without modifying the backend server code.

Benefits of React

Figure 7A: Classic and React tech stack comparison categorized by feature.

Feature	Classic	React / New Updates
Frontend	 Bootstrap  pug  jQuery  D3.js	 grommet  React  D3.js
Backend	 node  PostgreSQL  express  JS	 node  PostgreSQL  express  JS
Build Tool	 webpack	 Vite
Testing	 MOCHA  chai	 Vitest
State Management	Manual tracking of user interaction and input, and interface updates are tedious.	Automatic re-rendering of app based on user interaction and input, simplifying code.
Bug Fixes	Repetitive and complicated code makes bugs more likely and harder to find.	Modular code allows for easier discovery of bugs and fewer bugs created.

Figure 7B: React and Grommet are the tools that GRNsight has been migrated to.

Web Browser

```
<Button primary onClick={() => {}}  
  icon=<Add />  
  label="ADD ITEM"  
>/>
```

React JSX allows creation, access, and manipulation of DOM elements in the same place.

```
<Button  
  primary  
  onClick={() => {}}  
  label="Styled button"  
>/>
```

Card

Styled button

Grommet provides styling and theming via reusable components.



React is an open-source JavaScript library used for building frontend user interfaces for web and mobile applications (“React Foundations: About React and Next.js”). React is currently the industry-standard and one of the most popular web development frameworks because it simplifies the creation of interactive and dynamic web applications (“React Foundations: About React and Next.js”). React allows for the creation, access, and manipulation of DOM elements all in the same place (Figure 7B). React is declarative, meaning it handles rendering and updating the user interface automatically based on state changes, which reduces the need for developer-written manual DOM manipulation code. React’s most powerful feature is its state management.

React’s built-in way to share data across the app is by the use of state variables. These values can be read across the entire application using a React Context. When any of these state variable values change, React automatically calculates the most efficient way to re-render the app and update the screen without the manual manipulation of DOM elements (Figure 7B). In React, all code is separated into components, which are JavaScript functions that are independent and reusable. Therefore, React facilitates the creation of a modular codebase, which makes code easier to read and maintain while making it more difficult to cause bugs and easier to find bugs when they do exist (Figure 7A).

We chose to migrate GRNsight to React because doing so resolves many of the issues that the Pug, jQuery, and Bootstrap framework introduced. This version of the application is now called GRNsight React. React’s shared state directly addresses the problems of manual updates and inefficient data sharing inherent in the legacy framework. The use of a context ensures that any input values or user interactions are synced across the entire application simultaneously. Specifically, a context named GrnContext in App.jsx manages these global values. Then Context

Provider components are then used throughout the app to allow components to read and react to these values and any value changes.

Figure 8A: Adaptive variable in the GRNsight React codebase.

```
const [adaptive, setAdaptive] = useState(true);
```

Figure 8B: GRNsight React Viewport menu in the navigation bar which includes the Restrict graph to viewport selection.

Viewport Size

- Small (1104 x 648 pixels)
- ✓ Medium (1414 x 840 pixels)
- Large (1920 x 1080 pixels)
- Fit To Window

☐ Restrict Graph to Viewport

Zoom (25 - 200)

Figure 8C: GRNsight React Viewport menu in the sidebar which includes the Restrict graph to viewport selection.

View

Size

- ☐ Small (1104 x 648 pixels)
- ☒ Medium (1414 x 768 pixels)
- ☐ Large (1920 x 1080 pixels)
- ☐ Fit to Window
- ☐ Restrict Graph to Viewport

Using a context avoids the unnecessary complexity of passing the same properties to all the different child elements. For instance, the navigation bar (Figure 8B) and sidebar (Figure 8C) need to share and display the same state values. Using a context makes it much easier to both have a central source of data and to pass data between the components without passing several of the same properties through multiple levels of the component tree. A clear example of this is the adaptive variable, which is set to false when the "restrict graph to viewport" setting is enabled

(Figure 8A). When a user toggles this setting in the navigation bar or in the sidebar in React, the sidebar automatically re-renders with the updated selection value via the shared context.

Figure 9A: Classic code with two different listeners to detect when the “restrict graph to viewport” option is selected or deselected. #restrict-graph-to-viewport is the “restrict graph to viewport” button the navigation bar, and input[name=viewport] is the “restrict graph to viewport” button in the sidebar. The restrictGraphToViewport function is called whenever the “restrict graph to viewport”, or adaptive, value changes.

```

492     d3.select("#restrict-graph-to-viewport").on("click", function () {
493         var fixed = $("input[name=viewport]").prop("checked");
494         restrictGraphToViewport(fixed);
495     });
496
497     d3.selectAll("input[name=viewport]").on("change", function () {
498         var fixed = $(this).prop("checked");
499         restrictGraphToViewport(fixed);
500     });

```

Figure 9B: Classic code of the restrictGraphToViewport function, which shows how when the “restrict graph to viewport” value is changed, the display of that value has to be manually tracked and updated by the developer in both the navigation bar (#restrict-graph-to-viewport) and sidebar (input[name=viewport]).

```

466     var restrictGraphToViewport = function (fixed) {
467         if (!fixed) {
468             $("#restrict-graph-to-viewport span").removeClass("glyphicon-ok");
469             $("input[name=viewport]").removeProp("checked");
470             adaptive = true;
471             flexibleContainer = null;
472             center();
473         } else {
474             $("#restrict-graph-to-viewport span").addClass("glyphicon-ok");
475             $("input[name=viewport]").prop("checked", "checked");
476             adaptive = false;
477             $container.removeClass(CURSOR_CLASSES);
478             if (grnState.zoomValue > ZOOM_DISPLAY_MIDDLE) {
479                 grnState.zoomValue = ZOOM_DISPLAY_MIDDLE;
480                 updateAppBasedOnZoomValue();
481                 $container.removeClass(CURSOR_CLASSES);
482             }
483             width = $container.width();
484             height = $container.height();
485             d3.select("rect").attr("width", width).attr("height", height);
486             $(".boundingBox").attr("width", width).attr("height", height);
487             center();
488         }
489         updateAppBasedOnZoomValue(); // Update zoom value within bounds
490     };

```

This synchronization is much more difficult to accomplish in Pug/jQuery because the developer has to code listeners in the sidebar and navigation bar to detect specifically when the adaptive value changes. In GRNsight Classic, for the adaptive variable (Figure 8A), there are

two separate listeners (Figure 9): one to check when Restrict Graph to Viewport is clicked in the navigation bar (Figure 8B) and another to check when Restrict Graph to Viewport is clicked in the sidebar (Figure 8C). Whenever one of those values changes, the developer has to manually set the matching value in navigation bar or sidebar, and then the value has to get sent to the graph which is re-rendered manually as well (Figure 9B). It is important to note that these are two separate values, and there is no centralized adaptive variable that is being read. This tedious coordination between the navigation bar and sidebar has created a lot of bugs in the Classic app. However, in React, with state variables, if the value gets updated in either the navigation bar or the sidebar, React automatically calculates the most efficient way to re-render the app to reflect those value changes. There is no extra logic required to put out listeners since React detects that value change automatically and manages the synchronization across the navigation bar, sidebar, and graph.

The GRNsight React implementation relies on a Context Provider defined in App.jsx, which acts as a central hub for the application's state. This Provider wraps the component tree and broadcasts values to any “consumer” component that needs them, regardless of its position in the hierarchy. When a value within this context changes, React automatically triggers a re-render for only the components that are subscribed to that specific data. This reactive process ensures the UI stays synced with the latest inputs without the need for manual DOM updates. By decoupling data distribution from the component structure, we have created a more maintainable and performant platform.

Justification for Grommet UI Selection

In the Pug/jQuery/Bootstrap framework, an old and customized version of Bootstrap was used to style the web application. Bootstrap styles elements with pre-built CSS classes that can be added to individual DOM elements. To ensure that the user interface remained consistent with our existing design, we evaluated several popular styling frameworks, including Material UI, Tailwind CSS, and Grommet.

Material UI

Although Material UI is a robust and widely adopted framework, it is strictly opinionated and adheres closely to Google's Material Design specification (*Material UI - Overview*, 2025). This strict adherence creates significant default styling that would require extensive and tedious overriding to meet the unique aesthetic requirements of the GRNsight project. This inherent “out-of-the-box” look presents a major challenge when trying to faithfully preserve the pre-existing style and design of the GRNsight application. Ultimately, fighting against the library's built-in constraints would have cost more development time than building with a more flexible alternative. Rather than spending our efforts on feature development, we would have been forced to spend excessive time debugging CSS specificity issues. Consequently, we determined that a more utility-first or flexible component library would better support our need for a custom, data-heavy interface.

Furthermore, the overhead of importing the entire Material UI library would have introduced unnecessary bloat to our application's bundle size. While the library is powerful, many of its advanced features and complex animations were simply not relevant to the core functionality of GRNsight. This extra weight could have potentially slowed down the initial load

times, so by choosing a framework that better aligns with our specific needs, we kept the application lean without being weighed down by a rigid design system.

Tailwind CSS

We also considered Tailwind CSS for its utility-first approach to inline styling (*Tailwind CSS - Rapidly Build Modern Websites Without Ever Leaving Your HTML.*, n.d.). However, Tailwind lacks a comprehensive library of pre-built, accessible components in its free version. Access to high-level functional components is restricted to its paid tier, Tailwind UI. The free version of Tailwind styles elements by adding pre-defined CSS classes to individual elements, similar to Bootstrap. However, that way of styling elements can fill up the JSX code with CSS classes, making it harder to read. We knew we wanted to build a React app, so this meant we would have to implement more custom components on our end. Given our need for a rapid development cycle without additional licensing costs, Tailwind was deemed less efficient for this specific project.

Grommet UI

Ultimately, Grommet UI was selected as the frontend framework for its balance of flexibility and structure (*Grommet*, n.d.). Unlike Material UI and Tailwind CSS, Grommet provides accessible components with minimal default styling, allowing for a more streamlined integration with our current website's design. Grommet is easy to customize because the framework does not enforce a specific "brand" look. We could adapt components to our existing styles without the technical debt of overriding existing CSS rules. Grommet facilitates the implementation of global styles through a centralized theme object, ensuring visual consistency across the entire application. Individual components remain easily accessible for style overrides at the instance level, providing the precision necessary for a polished user experience.

Results and Discussion

API Updates to Connect to Backend

Figure 7A compares the tech stack for Classic and React, which I will discuss further below. To centralize the application's data layer, I consolidated all API calls into a dedicated `api.js` module, similar to GRNsight Classic. By structuring these as asynchronous functions, the rest of the application remains decoupled from the specific implementation details of the network requests; the components simply call a function and await a response without needing to manage the underlying HTTP logic. This module now serves as the exclusive bridge between our React frontend and the backend database server. As part of this transition, I migrated the “Load from Database” functionality away from legacy AJAX calls in favor of the modern `async/await` framework, providing a more readable and reliable way to handle asynchronous data requests.

Figure 10A: The Classic and React clients can make separate requests to the web API server to load a demo, import files, export a graph, or load genes/proteins from the GRNsight Network Database.

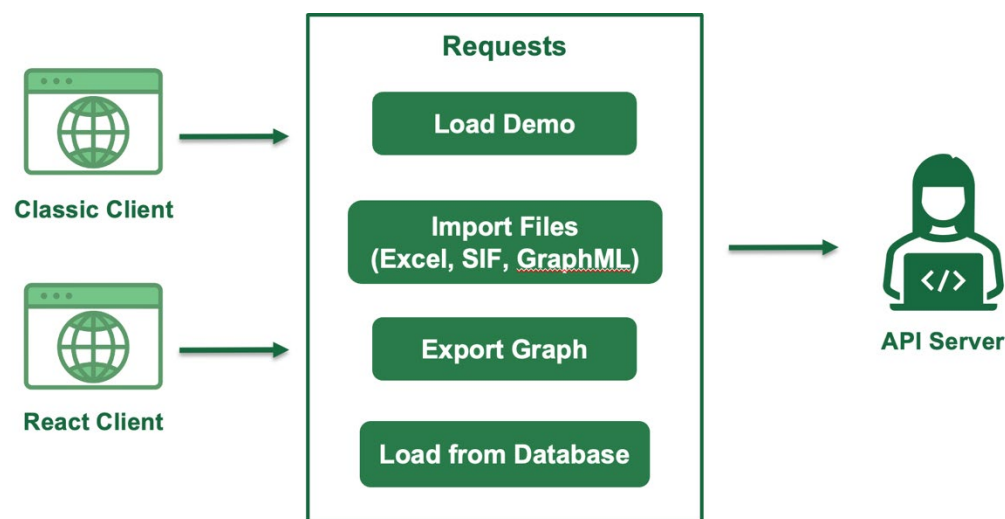
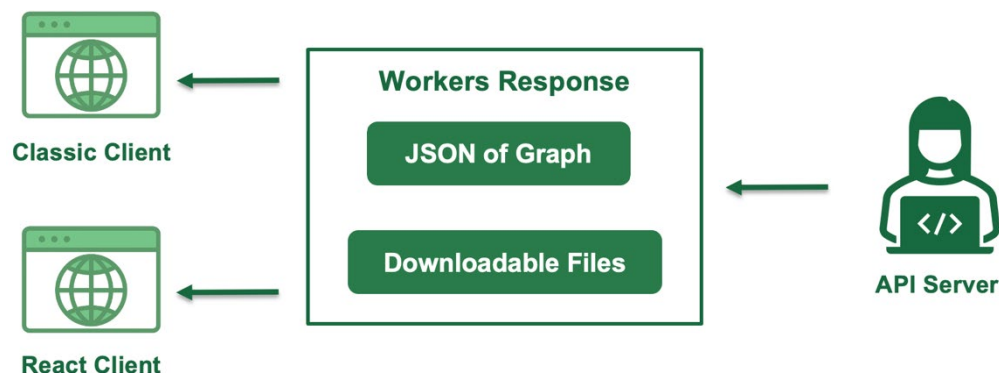


Figure 10B: The Classic and React clients can receive separate responses from the web API as the graph in JSON format or as downloadable files in the Excel, SIF, or GraphML formats.



Because GRNsight React is a Vite-based frontend application that resides within the existing project repository, we can leverage the pre-existing backend server code from Classic. This setup allows us to run the server locally or remotely while supporting two separate clients simultaneously: Classic and React. Both clients can send independent requests to the backend API (Figure 10A) and receive the appropriate responses (Figure 10B), which facilitates a smoother transition as we build out the new modular architecture. This dual-client approach ensures that we can continue to provide bug fixes for the original platform while iteratively testing the new application against the same reliable data source.

D3 for Force Graph Simulations

We highly customized D3.js in Classic to render the graph nodes and edges, and it contains all of the physics logic to render them as well (*What Is D3?*, 2024). The D3 customizations include the display of the rectangular nodes, node labels, end markers as arrowheads or dynamic vertical or horizontal repressor bars, edges as Bezier curves, dynamic viewport sizing, restriction of the nodes to the viewport bounds, and persistence of node positions after user dragging. In GRNsight React, I initially explored migrating to react-force-

graph, which handles the rendering logic internally by simply feeding data to a pre-built component. However, this library relies on Canvas, which renders everything to a pixel grid rather than discrete DOM elements. This presents a significant hurdle for our debugging process, as Canvas-based graphics are not inspectable with standard browser developer tools.

Furthermore, while modern browsers utilize graphics cards to boost Canvas performance, it remains a “black box” compared to the transparency of a component-based architecture. Since a core goal of GRNsight is to allow developers to easily troubleshoot and modify specific graph interactions, the lack of an inspectable DOM tree made Canvas an unsuitable choice.

The decision to avoid Canvas was also driven by our requirement for high-quality graphics. Unlike bit-mapped graphics that suffer from resolution loss when resized, SVG (Scalable Vector Graphics) maintains crisp lines at any scale. We currently have the ability to export graphs as SVG, PNG, and JPEG to ensure the platform remains versatile for various research needs. By rendering the graph as an SVG initially, we simplify the export process, as vector data is natively structured and easily converted into other formats. In a Canvas-based system, providing this same flexibility would have required redundant code and complex workarounds to generate high-quality outputs.

Ultimately, we chose to use D3 strictly for force simulation math while using React to handle the actual rendering of the SVG elements. Calculating the spatial physics, such as determining exactly how a dragged node pulls its connected neighbors, would have required immense manual labor to implement from scratch in pure React. Because we retained D3 for the force simulation logic, this made the React migration of our extensive D3 customizations much easier. Most of the logic remained unchanged and only required minor updates to accommodate React state variables and the newer version of the D3 library.

Updated Build and Testing Tools

Selecting the right frontend build tool was a critical step in modernizing the GRNsight architecture. GRNsight Classic uses Webpack for builds, which has slower re-renders and required extensive configuration to support the Pug/jQuery framework (Figure 7A). For GRNsight React, I evaluated several options and ultimately chose Vite as the primary build tool (*Why Vite*, 2026). Vite is a frontend build tool that has effectively replaced older utilities like Create React App by providing a significantly faster and leaner development experience with instant development builds and optimized production builds. It utilizes native ES modules, the standardized format for structuring and packaging JavaScript code for reuse, in the browser, which allows for near-instant server start times and lightning-fast Hot Module Replacement regardless of the application size (HMR) (*JavaScript Modules - JavaScript | MDN*, 2026; *Why Vite*, 2026). This performance boost is particularly beneficial for our project as we undergo the tedious process of migrating complex UI elements, as it allows us to see code changes reflected in the browser immediately.

The decision to implement Vite also aligns with our goal of creating a maintainable, modern codebase while navigating the transition from a multi-tier legacy system. With this migration, we decided to continue with a monorepo structure instead of migrating to a multi-repo structure to separate the frontend and backend. A monorepo is a unified repository with all the code for a project that includes code for different services, libraries, and applications while a multi-repo combines different repositories, which each contain the code for a distinct service, library, or application (Özkabadayı, 2024). Unlike more complex monorepo managers like Nx, Vite provides a straightforward, standalone structure that fits our current needs without adding unnecessary overhead. Beyond speed and local development, Vite provides a robust foundation

for our long-term deployment and CI/CD goals. By using Vite, we can easily manage our frontend builds independently, ensuring that our production assets are optimized for the performance levels required by data-heavy gene regulatory networks. While we considered moving to a multi-repo structure, the flexibility of Vite allows us to remain within our current monorepo while keeping our builds clean and separated from the legacy web-client. This strategic choice simplifies our workflow, as we can utilize the same GitHub Actions pipelines while preparing for a future where GRNsight React fully supersedes Classic. Ultimately, Vite offers the perfect balance of modern features and simplicity.

For testing, GRNsight Classic uses a combination of Mocha, a test runner, and Chai, which provides test assertions. Mocha and Chai do not have built-in capabilities to mock values, so Sinon.JS was additionally used to create mocks and stubs for testing application behavior and asynchronous interactions. In tandem with our shift to Vite, we selected Vitest as our primary testing framework because it was specifically engineered to work seamlessly within the Vite ecosystem (*Why Vitest*, 2026). Vitest is a modern, fast unit-test framework that shares the same configuration, plugins, and transformation pipeline as Vite itself. This shared architecture is a major benefit, as it ensures that the way our React components are bundled for the browser is exactly how they are processed during testing, leading to more consistent and reliable test results.

Vitest also offers significant performance advantages that complement our modular development workflow. It features a highly optimized “watch mode” that only re-runs tests related to the specific files changed, which dramatically speeds up the development cycle during complex refactorings. Additionally, it provides a powerful, built-in UI for visualizing test results and coverage, making it easier for our team to identify and resolve regressions in the GRNsight

functionality. Choosing a tool that is deeply integrated with our build system allows us to maintain high code quality without the friction of managing incompatible toolchains.

Feature Improvements

Sidebar Hide Panel Button

Figure 11A: Hide Panels button displayed in sidebar.

GRNsight React: v0.0.9 (05/01/2026)

Disclaimer: This version of GRNsight is currently under development and is unstable. For the most stable version, please go to the [GRNsight home page](#).

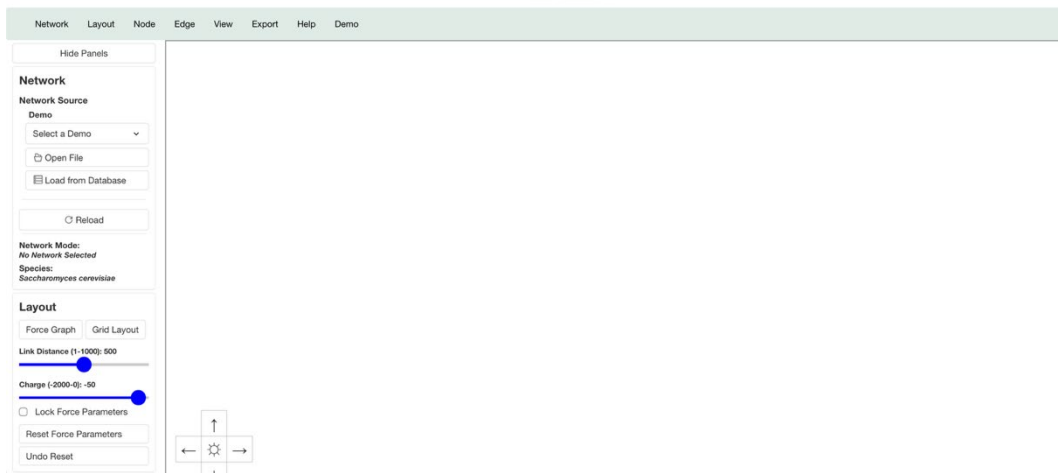
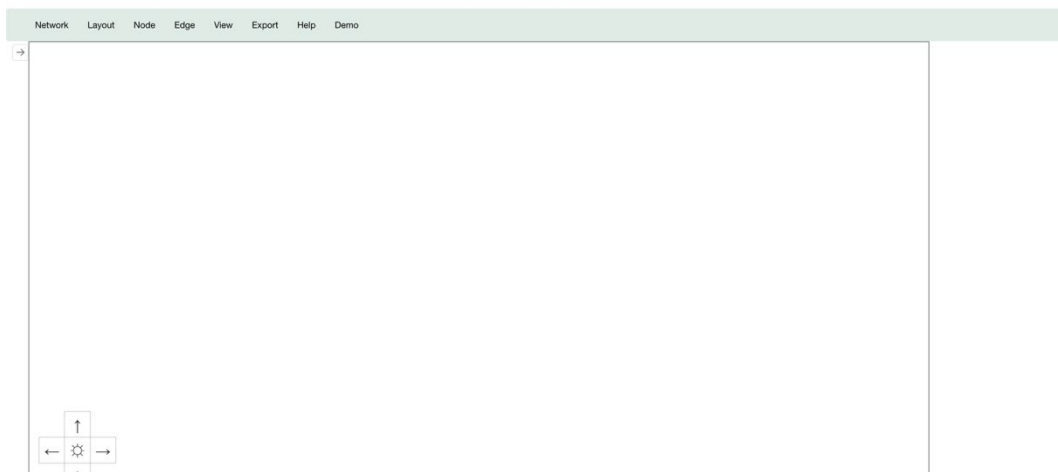


Figure 11B: After the Hide Panels button is clicked, then the sidebar collapses to the left and the Hide Panels button turns into an arrow button.

GRNsight React: v0.0.9 (05/01/2026)

Disclaimer: This version of GRNsight is currently under development and is unstable. For the most stable version, please go to the [GRNsight home page](#).



In GRNsight React, I implemented a “Hide Panels” button that is placed at the top of the sidebar to improve the graph viewing experience (Figure 11A). When a user clicks this button, it transitions into an arrow icon while the sidebar smoothly collapses toward the left side of the screen (Figure 11B). Clicking the arrow again expands the sidebar back to its original state, and the button label reverts to “Hide Panels.” This feature significantly enhances the GRNsight experience for users who want to maximize their screen space and want to ensure they can view and customize the graph more easily. This feature provides a more immersive feel by allowing the graph to take center stage with all of the sidebar functions still available in the navigation bar.

The implementation of this feature was made much more efficient by using Grommet’s built-in reusable components, such as the Collapsible component. This library already includes robust options for managing the opening and closing of elements, as well as handling the CSS transitions required for a smooth visual experience. The ease of adding such a dynamic feature highlights how the migration to React has led to direct and tangible improvements to the user experience. Although the overall migration process to a new framework is often tedious and requires significant effort, the separation of code into smaller, modular components in React makes these UI enhancements much more manageable. This architectural shift significantly lowers the barrier to entry for future refinements. Ultimately, by breaking the application down into independent pieces, we have opened up the possibility for even further UI enhancements that were previously too difficult to implement. The modularity of React ensures that our interface can now evolve as quickly as the needs of our users.

To control the sidebar’s behavior, I created a simple state variable that tracks whether the panel is currently open or collapsed. When the “Hide Panels” button is clicked, this state updates accordingly, and React automatically triggers the necessary re-renders to reflect that change

across the UI. In the legacy Pug and jQuery framework, achieving this same result would have been far more difficult and prone to bugs. I would have had to manually manipulate the DOM, likely by adding or removing CSS classes across multiple disconnected elements while ensuring the animation timing stayed in sync.

Resolution of Long-Standing Arrowhead Bug

Figure 12A: Classic Arrowheads with space between the arrowheads and the right perimeter of the node.

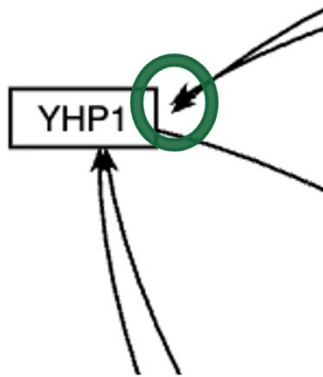
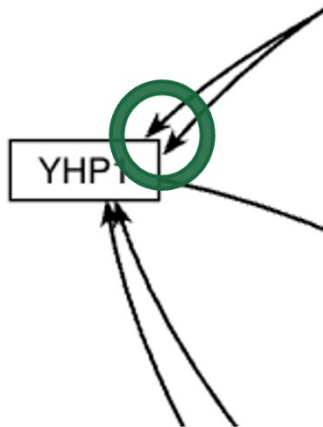


Figure 12B: React Arrowheads touching the perimeter of the node.



In GRNsight Classic, a long-standing visual bug resulted in an awkward gap between the arrowhead and the right-side perimeter of a node. While this was a noticeable aesthetic issue, it was never prioritized for fixing because it did not interfere with the core functionality of the network simulations. During the migration to React, I took the opportunity to conduct a deep

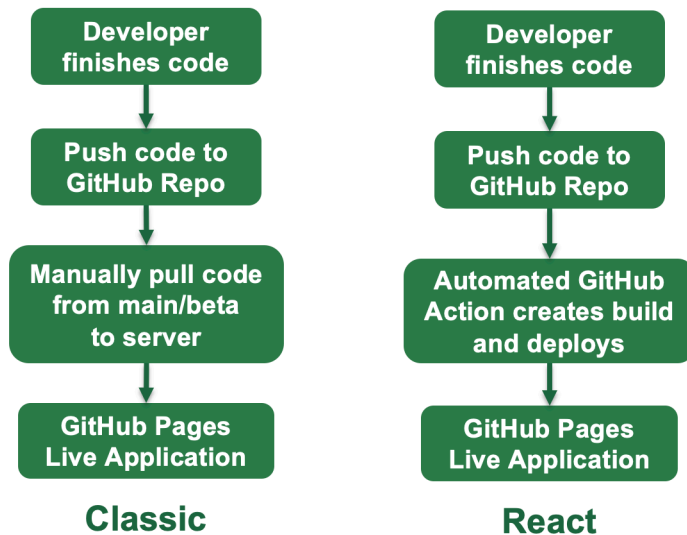
dive into the legacy logic responsible for calculating arrowhead positions. I discovered a hard-coded variable named `LEFT_ADJUSTMENT` with a value of 7 that was being added specifically when an arrow approached the right side of a node. By isolating this specific line of code, I was able to modify the calculation so that this unnecessary offset is no longer applied.

Before finalizing the fix, I carefully considered whether this gap was an intentional design choice from a previous development cycle. I consulted with my research professors to see if there was a specific biological or UI use case that required extra spacing on the right side of a node. They confirmed that there was no current functional reason for the adjustment, which allowed me to confidently remove the legacy logic. This discovery perfectly illustrates how a migration serves as a high-level code review for the entire application. While the process of porting code is often tedious, it provides a rare chance to scrub away years of “technical debt” and minor visual inconsistencies.

This experience also highlights why it is vital to manually review and understand pre-existing functionality during a framework shift. If I had simply used an AI prompt to translate the legacy code into React syntax, this bug would have been ported over without anyone noticing. By taking the time to read through the code and truly understand the geometry behind the arrowhead logic, I was able to isolate and resolve a bug that had persisted for years. Using a migration as an opportunity for critical review ensures that the new version of the app is not just a copy, but a genuine improvement. Ultimately, these small refinements contribute to a more polished and professional user experience for the scientific community.

Automated Deployment

Figure 13: Comparison between the Classic and React deployment steps.



I have implemented continuous integration (CI) and automated deployment of GRNsight React to GitHub Pages using GitHub Actions. This modernization represents a significant functional improvement over the legacy deployment workflow used for GRNsight Classic. There are four steps to deploy, and the difference between Classic and React is the third step in deployment (Figure 13). In Classic, we have to login to the server and manually pull code from the main or beta branches in order for the app to deploy to GitHub Pages. In React, this has been automated with a GitHub Action so that when code is pushed to the main or beta branches, a build is automatically created and deployed to GitHub Pages. This transition highlights how migrations provide an ideal opportunity to introduce sophisticated improvements to DevOps processes, ultimately making the development lifecycle more efficient and less prone to manual error.

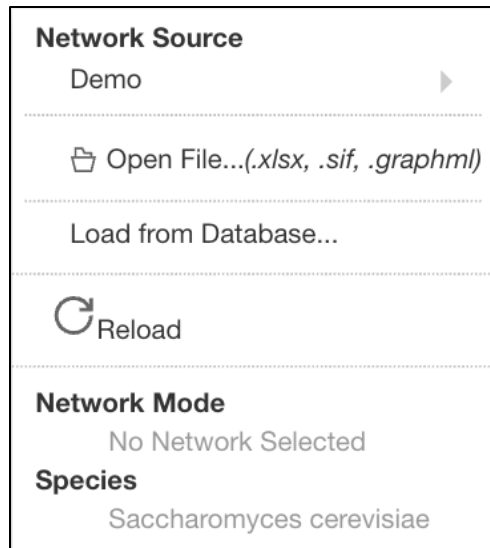
Modular Code Components

The migration to React has significantly improved the modularity of the GRNsight codebase. Previously, the legacy system relied on files containing thousands of lines of code, which were difficult to navigate and maintain. React has made it much easier to separate these monolithic files into smaller, specialized components and shorter files. This organizational shift drastically enhances the readability of the code and simplifies long-term maintenance. By breaking the application down, developers can now focus on isolated pieces of the interface without being overwhelmed by unrelated logic.

The most significant impact of this reorganization is illustrated through the dramatic reduction in file complexity, especially regarding the migration from the original `graph.js` from Classic which handled all the logic and rendering of the graph display and D3 customizations. In Classic, the `graph.js` file was a staggering 2,071 lines long, making it nearly impossible to debug efficiently. Following the migration, the primary `Graph.jsx` component has been reduced to just 688 lines. This significant reduction in size shows that moving to a component-based architecture has made the project much more manageable and less prone to the errors. In GRNsight React, `Graph.jsx` serves as the main parent component where D3 calculations and SVG rendering occur, while `ScaleAndScroll.jsx` independently manages the d-pad and zoom inputs. These components act as self-contained units, which prevents changes in one part of the UI from accidentally breaking another. We also divided the original `graph.js` file into a suite of organized helper files, separating the UI logic from math calculations or data processing. We created scripts such as `graphHelpers.js` for calculating node and edge properties, `markerHelpers.js` for arrowhead logic, and `restrictGraphToViewportHelpers.js` for node boundary calculations when Restrict Graph to Viewport is enabled. Additionally, files like `constants.js`,

theme.js, and viewportHelpers.js now handle global settings and layout initializations independently. This separation ensures that specific mathematical logic is kept separate from the visual rendering code, making the entire system more readable and understandable.

Figure 14: Network menu in the navigation bar that displays a DottedLine component with a 95% width after the demo button and open file button. The DottedLine component has a 100% width after the load from database button and reload button.



Another practical benefit of this modularity is the creation of reusable components across the application. For example, I was able to transform the DottedLine element into its own independent component that appears several times throughout the navigation bar dropdown menus (Figure 14). This component is highly flexible because it can accept a width property to customize its appearance in different contexts. If a specific width is provided, such as 95%, the component adjusts accordingly, otherwise, it defaults to 100%. Creating such customizable and reusable UI elements was simply not possible with the original Pug and jQuery framework.

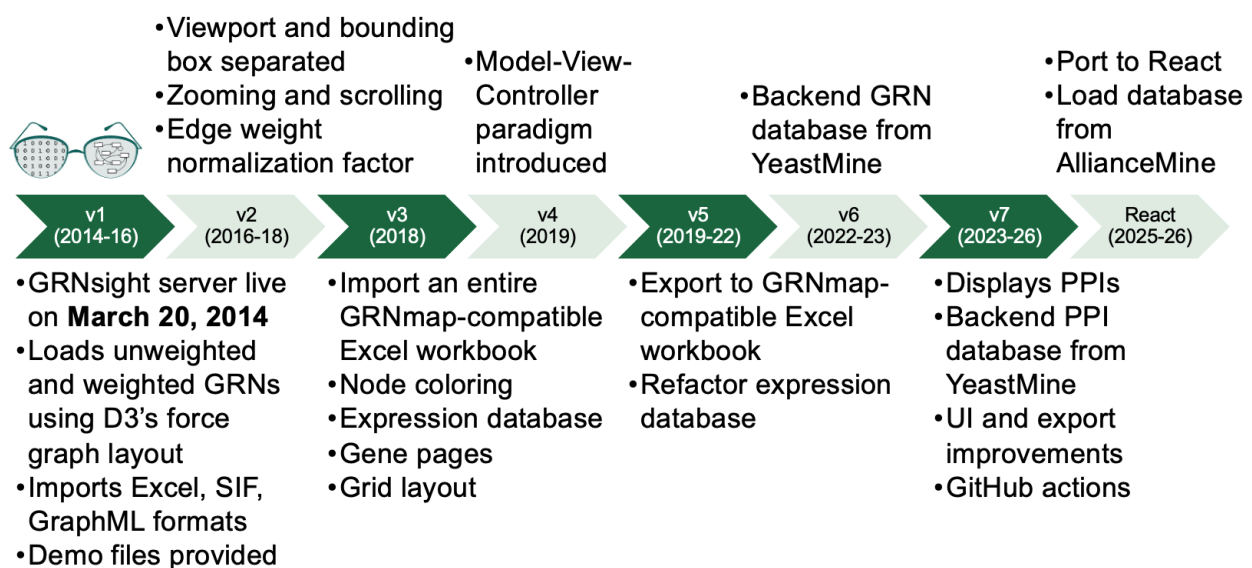
React also allows for the seamless integration of logic directly into DOM elements, which further streamlines the development process. In the DottedLine component, I am able to dynamically set the margin using a ternary conditional statement. This logic checks if a width

property was provided; if so, it sets the margin to undefined, and if not, it applies a vertical width of 9px. This level of inline conditional styling is a major advantage that Pug files do not support. Integrating this logic directly into the component ensures that the styling remains consistent and reactive to the data it receives.

Ultimately, the ability to embed functional logic within the structure of the UI is one of the primary benefits of migrating to React. In the previous framework, these types of adjustments would have required separate scripts to manually manipulate the DOM after it had already rendered. With React, the logic and the element are bundled together, reducing the risk of synchronization errors. This approach results in a much more cohesive development experience where the code reflects the intended behavior of the UI. This transition ensures that the GRNsight platform is both robust and adaptable for future feature sets.

Migration Timeline

Figure 15: Timeline of GRNsight versions based on features and year.



GRNsight has been an ongoing endeavor led by successive teams of undergraduate researchers since its inception in 2014 (Figure 15). New features have been implemented every one to two years, resulting in a complex and mature application. We strategically deferred the migration to React until now because we remain committed to maintaining the Classic application with consistent bug fixes and new feature updates. While code migrations offer substantial performance benefits, they require a significant investment of laboratory hours to ensure a successful transition. This large-scale move to React proved most effective when at least one team member was dedicated exclusively to React-specific tickets and architectural issues. Having the resources to assign a focused developer to this task was the primary catalyst for our recent progress.

Throughout its history, the GRNsight team has varied in size from as few as two to as many as seven researchers. Currently, our team is sufficiently staffed to maintain GRNsight Classic while providing the necessary bandwidth for me to focus entirely on the React migration. By balancing the immediate needs of our users with long-term technical goals, we were able to commit to this modernization without compromising current support, which underscores the importance of having dedicated roles during major technological shifts.

A pivotal effort in 2019 to transition the project toward a Model-View-Controller paradigm laid the essential groundwork for our current success. This early shift led to the creation of the global `grnstate` object, which was designed to emulate the behavior of state variables found in React. While that effort was not a full framework migration, it moved the codebase's logic toward a "React-like" structure well before the actual transition began. By adopting these patterns early, the team demonstrated a forward-thinking approach to software

sustainability. This incremental change showed that even partial migrations can yield long-term dividends for a project's evolution.

The existence of `grnstate` significantly simplified the actual migration process because I could directly translate its properties into the modern `GrnContext`. Since functions throughout the application were already written with the assumption of a centralized data object, I saved a considerable amount of time on logic refactoring. This “step-in-between” allowed the application to evolve its data management strategy without a total system failure. It serves as a clear example of how writing code with future migrations in mind can dramatically reduce technical friction years later. Ultimately, this proactive architectural planning ensured that the transition to a component-based system was as seamless as possible.

Agile Program Management Strategy

I used the Agile development strategy to manage the milestones on this project. For the design, I already knew that we were going to replicate the design of `GRNsight Classic`, which simplified much of the initial planning process. I started by creating larger high-level tickets such as “Create frontend in React” and “Port the graph display to React.” As development progressed, I scaffolded these into smaller and more specific tasks such as “Move Controls and Zoom Scrollbar” and “Drag Graph Background.” Some tickets also functioned as spikes, where additional investigation and experimentation were needed before implementation. For example, the ticket “Determine strategy for porting to Vitest” involved researching and evaluating possible approaches for testing in general before completing the actual migration work to port to Vitest. As I worked through different issues, I continuously created additional tickets when new requirements, implementation details, or bugs were discovered. Starting with broader tickets

helped establish the overall project direction, while later breaking them into smaller tasks made it easier to address specific problems and refine features incrementally. Development followed an iterative cycle of implementing features, testing functionality, identifying issues, and making revisions as needed (Kavlakoglu, 2025). This approach differs from the waterfall strategy, which follows a more linear process of design, ticket creation, and feature implementation from beginning to end (Kavlakoglu, 2025).

I organized the project milestones with GitHub Projects, which allows for planning and tracking work on GitHub (*About Projects*, 2026). I used GitHub Projects in the Board view, and I found the board-style workflow intuitive and effective for visualizing milestones, tracking progress, and organizing development tasks into the five different stages of backlog, in progress, pull request (PR) created, review requested, and done. Tickets in the backlog stage represented planned features, bugs, or improvements that still needed to be prioritized and implemented. Once development began on a task, the ticket moved to in progress, where active coding, debugging, and testing took place. After completing an implementation, the task moved to pull request (PR) created, indicating that the code changes had been submitted for integration into the main codebase. The ticket then advanced to review requested, where feedback, testing, and code review were performed by my research professors to verify correctness of the feature. Finally, once the feature or fix was approved and merged, the ticket moved to done, marking the completion of that milestone.

Artificial Intelligence Usage

I utilized generative AI during the initial stages of this project to explore its potential for automating the migration process. Specifically, I wanted to see if a large language model could

successfully port the complex graph portion of the application over to React. The AI-generated implementation, however, failed to account for the extensive customizations that are central to the GRNsight interface. Rather than adopting a modern React architecture, AI maintained the outdated `grnstate.js` structure by treating `grnstate` as a static object instead of converting it into a functional `useState` variable. This approach demonstrated a lack of understanding regarding the underlying state management improvements we aimed to achieve with this migration.

The complexity of the `graph.js` file presented a significant hurdle for automated tools due to its length and intricate dependencies. At thousands of lines long and featuring a high volume of external imports, the file contains specialized logic that requires deep contextual knowledge to navigate. For example, the AI implementation replaced our refined Bezier curves with basic straight lines for all edges. It also failed to correctly render self-regulating edges, which we require to originate on the right side of a node and terminate at the bottom without the arrowhead disappearing behind the element. These nuances are vital but were overlooked by the generative AI tools that I used. More expensive, paid models may have been able to migrate the graph code more correctly.

Ultimately, this exercise showed that the GRNsight project involves too much specialized customization for an AI to handle without significant human intervention. The intricate display logic and graph behaviors we have developed over the years require a developer's eye to ensure the resulting code is accurate. Relying solely on automated porting would likely introduce unnecessary “bloat” or redundant code into our repository, complicating future maintenance. Human involvement remains a necessity to verify that every line of code serves a specific functional purpose and that the integrity of our code is preserved during this migration.

While I found that generative AI could not handle a full-scale automated migration, it proved to be an invaluable tool for navigating and auditing the legacy codebase. Because there are Classic files that are thousands of lines long, isolating the specific logic for certain features is a time-consuming manual task. I used AI as a specialized search engine, asking prompts such as, “Where does the code pertaining to generating a network or loading from the database appear in web-client-classic?” Once the agent identified the relevant files, I could review them to understand how they functioned both in isolation and in coordination with the rest of the app. This allowed me to perform a high-level code review of the pre-existing logic before I attempting to port it over to React.

Once I understood the logic of the files needed to be ported over, I used AI to assist in the actual translation of specific functions to modern JavaScript and React standards. For example, I asked it to port over the upload logic and API functions for the “Generate Network” feature. After AI provided a draft, I carefully reviewed the output and compared it with the legacy code to ensure the core logic remained intact or to identify potential improvements made during the port. This iterative process ensured that the “heavy lifting” of the migration was done accurately.

However, I noticed that AI struggled significantly when attempting to port over frontend and UI components. When I asked it to port the “Generate Network” modal, the resulting interface did not match our established design. This was likely due to our project's unique combination of custom CSS and older Bootstrap styling, which is difficult for AI to interpret without extensive context. These limitations became especially apparent during the graph migration because the AI agent simply could not replicate the visual nuances of our network visualizations. Consequently, I had to handle most of the graph porting manually, requesting AI assistance only for small, isolated chunks of logic.

Even with these frontend challenges, AI remained helpful for pinpointing the location of complex visual logic within the legacy system. I frequently used it to answer specific architectural questions, such as “Where do the arrowheads get created and customized?” by pointing it toward our original D3 implementation. Having an agent that could instantly parse through thousands of lines of code to find a single variable or function saved an enormous amount of research time. This enabled me to focus my energy on the creative and technical problem-solving required to bring GRNsight into a modern component-based architecture.

Completed and Pending Classic Features

In this migration, most of the Classic features were ported over, but there are still some pending features that are remaining. Below is a description of the different features that remain to be ported over to React.

Completed Features	Pending Features
• Main navigation bar and sidebar interface and force graph display. • Network menu, Open Excel, SIF, GraphML files, Load Demo files, Load from Database. • Layout menu: link distance and charge sliders. • Edge menu: enable/disable edge coloring, set gray threshold. • View menu: change viewport size (small, medium, large, fit to window), restrict graph to viewport, d-pad, zoom, scroll, pan, grabbing whole graph, moving individual nodes. • Help menu links.	• Network menu: reload button. • Layout menu: grid layout. • Node menu: enable/disable node coloring, node coloring dropdown menu, average replicate values checkbox, log fold change max value. • Edge menu: display edge weights, edge weight normalization factor, show gray edges as dashed. • Export and print.

Future Work

Looking ahead, the primary focus for the next development cycle involves completing the migration of the node coloring and warnings modules for the open-files feature. This phase includes the critical task of porting over hundreds of existing open-file tests, which function as a separate validation layer from standard code coverage by specifically testing the end-to-end functionality of the file-loading process. Once these core features are migrated over, the team plans to integrate GRNsight React directly into the main GRNsight website. This integration will allow users to select the React version as a distinct option alongside the current “Home” (main) and “Beta” versions, moving away from its current status as a separate, standalone URL. Ultimately, the long-term roadmap for the project also involves transitioning the React codebase into the primary production version as GRNsight Classic is gradually phased out.

Conclusions

The successful migration of GRNsight from a legacy Pug/jQuery/Bootstrap architecture to a modern React/Grommet framework represents a transformative milestone for the project’s long-term sustainability. By transitioning to a component-based system, we have replaced monolithic, thousand-line files with a modular architecture that separates the user interface from mathematical logic. React has improved code maintainability because of its automated user input tracking, built-in ways to store data throughout the app, and automatic re-rendering of the app. This structural shift not only drastically reduces file complexity and enhances readability, but it simplifies the development process for future generations of undergraduate researchers. The adoption of modern tools like Vite, Vitest, and GitHub Actions further professionalizes our

development lifecycle, providing a high-performance environment that supports rapid iteration and automated deployment.

Beyond the technical improvements to the codebase, this migration served as a high-level code review that allowed us to resolve long-standing visual bugs and refine the user experience. By leveraging React's context for global state management and utilizing a hybrid D3-React approach for graph rendering, we have maintained the sophisticated scientific functionality of GRNsight Classic while introducing a more responsive and maintainable UI. We prioritized the integrity of our vector graphics, ensuring that researchers can continue to export professional-grade, publication-ready visuals in multiple formats. These enhancements ensure that GRNsight remains a robust and adaptable tool for biologists who model GRNs.

Ultimately, this project highlights the immense value of proactive architectural planning and the necessity of human oversight when coding with generative AI tools. While the migration process required a lot of time and effort, the resulting platform is significantly more flexible and open to future UI enhancements that would have previously been extremely difficult to implement. We have successfully bridged the gap between the 12 years of legacy development and modern web standards, creating a stable foundation for growth. As GRNsight continues to evolve, this new architecture ensures it will remain a scalable and efficient resource for visualizing GRNs and PPIs.

Acknowledgements

I would like to thank my research professors, Dr. John David N. Dionisio and Dr. Kam D. Dahlquist, for their mentorship and guidance throughout my 5 semesters on the GRNsight team. I would also like to thank the rest of the Fall 2025 - Spring 2026 GRNsight undergraduate

research team, Ngoc Kim Ngan Tran, Milka Y. Zekarias, Amelie T. Dinh, Jia S. Garcia, Cindy L. Tong, and Alex J. Miller as well as Nikki C. Chun and the previous developers on the GRNsight project. Finally, I would like to thank the LMU Department of Biology and the HHMI STEM Research Learning Community for their support and funding of the GRNsight project.

Additional Information and Declarations

Data Deposition

The following information was supplied regarding data availability:

GitHub code repository: <https://github.com/dondi/GRNsight>

GRNsight React web application: <https://dondi.github.io/GRNsight/react-thesis-4081/>

GRNsight Classic web application: <https://dondi.github.io/GRNsight/>

References

About Projects. (2026, March 18). GitHub Docs. Retrieved May 13, 2026, from

<https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>

Bhattacharyea, A. (2025, October 17). Why I still use JQuery in 2025 (and when not to) |

Docker. *Docker*. Retrieved May 13, 2026, from <https://www.docker.com/blog/why-i-still-use-jquery-2025/>

Brody, L. (n.d.). Gene expression. *National Human Genome Research Institute*. Retrieved April

25, 2026, from <https://www.genome.gov/genetics-glossary/Gene-Expression>

Dahlquist, K.D., Fitzpatrick, B.G., Camacho, E.T., Entzminger, S.D., and Wanner, N.C. (2015)

Parameter Estimation for Gene Regulatory Networks from Microarray Data: Cold Shock

- Response in *Saccharomyces cerevisiae*. *Bulletin of Mathematical Biology*, 77(8), 1457-1492, published online September 29, 2015. DOI: 10.1007/s11538-015-0092-6.
- Dahlquist KD, Dionisio JDN, Fitzpatrick BG, Anguiano NA, Varshneya A, Southwick BJ, Samdarshi M. 2016. GRNsight: a web application and service for visualizing models of small- to medium-scale gene regulatory networks. *PeerJ Computer Science* 2:e85 <https://doi.org/10.7717/peerj-cs.85>
- Farooq, Q. U. A., et al. (2021). Protein-protein interactions: Methods, Databases, and Applications in Virus-host Study. *World Journal of Virology*, 10(6), 288–300. <https://doi.org/10.5501/wjv.v10.i6.288>
- Grommet. (n.d.). Grommet V2. Retrieved May 13, 2026, from <https://v2.grommet.io/>
- JavaScript Modules - JavaScript | MDN*. (2026, April 4). Retrieved May 13, 2026, from <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>
- Kavlakoglu, E. (2025, November 17). Agile vs. Waterfall. IBM Think Topics. Retrieved May 11, 2026, from <https://www.ibm.com/think/topics/agile-vs-waterfall>
- Material UI - Overview*. (2025, November 26). Material UI. Retrieved May 13, 2026, from <https://mui.com/material-ui/getting-started/>
- Methvin, D. (2014, January 13). *The State of JQuery 2014 | Official JQuery blog*. Retrieved May 13, 2026, from <https://blog.jquery.com/2014/01/13/the-state-of-jquery-2014/>
- Özkabadayı, K. (2024, May 5). *Choosing Between Monorepo and Multi-Repo Architectures in Software Development*. Medium. Retrieved May 13, 2026, from <https://medium.com/@kazimozkabadayi/choosing-between-monorepo-and-multi-repo-architectures-in-software-development-5b9357334ed2>

RCSB PDB-101: Molecule of the Month – Vitamin D Receptor. (n.d.). RCSB Protein Data Bank. Retrieved May 13, 2026, from <https://pdb101.rcsb.org/motm/155>

RCSB PDB-101: Molecule of the Month – Cyclin and Cyclin-dependent Kinase. (n.d.). RCSB Protein Data Bank. Retrieved May 13, 2026, from <https://pdb101.rcsb.org/motm/236>

React Foundations: About React and Next.js. (n.d.). Next.js. Retrieved April 25, 2026, from <https://nextjs.org/learn/react-foundations/what-is-react-and-nextjs>

Tailwind CSS - Rapidly build modern websites without ever leaving your HTML. (n.d.). Tailwind CSS. Retrieved May 13, 2026, from <https://tailwindcss.com/>

What is D3? (2024, October 18). D3 By Observable. Retrieved May 14, 2026, from <https://d3js.org/what-is-d3>

Why Vite. (2026, March 11). Vite. Retrieved May 13, 2026, from <https://vite.dev/guide/why>

Why Vitest. (2026, April 8). Vitest. Retrieved May 13, 2026, from <https://vitest.dev/guide/why.html>